

WorkoutLang — D5: Retrospective & Self-Assessment (Part 1)

Gebze Technical University | Spring 2026 | Due: 8 May 2026

D5 — Part 1 Retrospective

What Is Working

The lexer and parser are complete and functioning correctly. All three valid test programs parse successfully and produce well-formed, readable ASTs. All five malformed programs are rejected with informative error messages that include the exact line number and a description of what was expected. The token types for domain-specific literals (DURATION_LIT with suffixes s/min/hr, and WEIGHT_LIT with suffixes kg/lb) work correctly and are never confused with plain identifiers. The set statement — the central domain-specific construct of WorkoutLang — parses cleanly. The expression grammar implements the correct left-to-right associativity and the six-level precedence hierarchy defined in D1. The project compiles with no warnings under gcc -Wall -Wextra -std=c11.

What Is Hard

The most technically difficult part was getting the expression parser right. The AI-generated EBNF (documented in Entry 2 / Experiment E1) used left-recursive rules that are incompatible with recursive descent parsing. Converting those rules to the iterative Sebesta style and verifying that the resulting parser was left-associative required writing dedicated test expressions such as '4 - 2 - 1' and tracing the AST to confirm the parse tree was '(4-2)-1' and not '4-(2-1)'. Error recovery in the parser is also imperfect: when one error cascades, subsequent error messages can refer to the wrong token. This is acceptable for Part 1 but will need improvement.

What I Plan to Fix in Part 2

Type checker: Section 4.5 of D1 is not yet written. Part 2 requires a complete type system with strong typing, coercion rules, and type equivalence decisions. I plan to implement a symbol table in C and a two-pass type checker (first pass for declarations, second pass for statement bodies).

Interpreter: The interpreter needs to respect the static scoping rule declared in §4.6. I will implement an environment chain (a linked list of scopes) where each function call or block entry pushes a new scope and return/exit pops it.

Error recovery: The parser currently continues after errors but sometimes produces misleading cascading messages. I will add a synchronization mechanism (skip tokens until a safe recovery point such as '}' or a keyword) to reduce noise after the first error.